

How OrcaVault Put an AI Agent On-Chain with ERC-8004

A plain-language, step-by-step walkthrough — from the MCP wrapper to the payment setup to all three on-chain registries. Every address and transaction below is public and verifiable on-chain.

The one-paragraph version

OrcaVault is a service that stores files permanently on the Lightchain blockchain. We wrapped it as an **MCP tool** so that AI agents — not just people — can use it directly. Because an AI agent has no wallet and can't pay or sign for itself, we gave it a **pre-paid agent wallet** that the server signs with internally. Then we gave that agent a real **ERC-8004 on-chain identity**, added the **Validation** and **Reputation** registries the standard defines, and **dual-registered** it on Base so Ethereum-side tools can discover it too. The result: a working, paid AI service that also speaks the emerging "trust layer for AI" standard — and anyone can verify every piece of it on-chain.

Part 1 — The problem, and why it's interesting

The Ethereum Foundation has been public about a big idea: as AI agents start doing real work, they'll need a way to **identify themselves, build trust, and pay each other** — and a blockchain is a natural place to anchor that. The heavy AI computing stays off-chain; the chain acts as the verification and coordination layer. The standard for this is **ERC-8004** ("Trustless Agents").

OrcaVault already had the hard part most projects don't: **a live, paid product**. What it didn't have yet was a way for AI agents to use it, and an on-chain identity that speaks the standard. This guide is how we closed that gap.

Two honest framing notes up front, because they matter:

- An ERC-8004 identity gives an agent **discovery**, not automatic **trust**. Being findable is not the same as being proven trustworthy.
 - Where our current setup is self-attested rather than independently verified, we say so plainly below. We don't overclaim.
-

Part 2 — The MCP wrapper (letting an AI agent use OrcaVault)

MCP (Model Context Protocol) is the standard way to hand an AI agent a set of tools it can call. We wrapped the OrcaVault relay as an MCP server exposing **five simple tools** that cover the whole loop — check, store, confirm, retrieve:

Tool	What it does
check_access	Is this wallet allowed to upload, and what's the fee?
relay_status	Is the relay healthy and funded?
store_memory	Save a file on-chain. Returns a <code>memoryId</code> .
get_memory_status	Is memory <code>#N</code> assembled and ready yet?
get_memory	Get memory <code>#N</code> 's bytes back.

The key trick: the pre-paid agent wallet

An AI agent has no wallet, so it can't pay LCAI or sign the relay's ownership message on its own. We solved that like this:

1. A dedicated **agent wallet** is registered with the relay **once**.
2. The MCP server **holds that wallet's key and signs uploads internally** — the agent never sees the key. It just calls the tools.
3. The agent wallet needs **no ongoing balance**: the relay pays all the gas. The only cost is the one-time registration.

So from the agent's point of view, storing a file on a blockchain is as simple as calling `store_memory(...)` and getting back a `memoryId`. All the wallet, signing, and gas complexity is hidden.

Part 3 — The payment setup (how an agent pays to get in)

OrcaVault's model is deliberately simple and matches what a human user does on the site today. It's a **one-time registration unlock, not a per-upload fee** — once a wallet is unlocked, its uploads are covered and the relay pays the gas. You can pay in **either of two tokens**, both live today:

- **2 LCAI** (native Lightchain) — transfer 2 LCAI to the payment address, then register.
- **or 1 USDC (bridged)** — bridge USDC onto Lightchain via a Hyperlane warp route (Base collateral → Lightchain synthetic USDC), transfer at least 1.0 USDC to the same payment address, then register with that transfer's tx hash.

Both options settle to the same address on the same chain:

- **Payment address (LCAI or USDC):** `0xbb0ab4c9E15a20661CA0C4d2b6f5D32A7EdF7646`
- **Bridged-USDC token contract (Lightchain):** `0x1113E23397b4398eF1A46A9A7f27C2241527b003` (6 decimals)
- **Chain:** Lightchain AI Mainnet, **chain ID 9200**.

This is the piece ERC-8004 deliberately leaves out. The standard handles identity, reputation, and validation, but **explicitly does not define payments** — it leaves monetization "to higher-level protocols." OrcaVault is that higher-level piece: it already had a working paid service, so it fills the exact gap the standard punts on.

A note on cost as a strength: ERC-8004's whole vision is lots of small agent-to-agent payments. On Ethereum mainnet, gas eats those alive. On Lightchain, gas is near-zero — a full file uploads for a fraction of a cent — which makes a near-free chain arguably a better fit for high-frequency agent payments.

Part 4 — The agent's on-chain identity (ERC-8004 Identity Registry)

This is the first of the three ERC-8004 registries, and the core "speak the standard" step.

What it is: the Identity Registry is a smart contract that mints an **ERC-721 NFT** as the agent's ID. That NFT points to a small JSON "**agent card**" describing what the agent does, its endpoints, and its payment address. On-chain NFT = the anchor; off-chain JSON = the context. Discovery tools read the registry and the card.

How we built it:

1. **Deployed the official reference Identity Registry** contract (from the ChaosChain `trustless-agents-erc-ri` reference implementation — not a hand-rolled contract, so third-party indexers recognize it as standard-compliant) to Lightchain (chain 9200).
2. **Wrote the agent card JSON** — name ("OrcaVault MCP"), description, service endpoints (MCP + web), payment address, and chain — and hosted it at a stable URL.
3. **Minted one agent NFT** for the OrcaVault MCP service, owned by the agent wallet.

Verifiable values:

- **Identity Registry:** `0x8cA5cc5037aF83762312b464e04426795d62CC40`
- **Deploy tx:** `0xfe3473a89fde0daeb37622dcb0f2867aa48ac8dcfbf926fa9ea394b81b38b05c`
- **Agent ID (tokenId):** `1`
- **Register (mint) tx:** `0x3c03ddd5fd402ba7c1b853b0edf63bdfa805c0558c9631b71016a42d646504b`
- **Agent NFT owner:** `0xBF78164b6626ea8096C4B6b268b0b70f86f66893`
- **Agent card URL:** <https://orcavault.win/agents/orcavault-mcp-card.json>

- **agentRegistry (CAIP format):** `eip155:9200:0x8cA5cc5037aF83762312b464e04426795d62CC40`

Part 5 — Verifiable proof of work (ERC-8004 Validation Registry)

This is the second registry — the "verification layer" piece specifically.

What it is: the Validation Registry lets an agent **request** an independent check of a piece of its work, and lets a **validator** respond with a score of 0–100 that's recorded on-chain. For OrcaVault, the "work" is a stored file, and the natural check is: does the content hash recorded on-chain match what was actually stored?

How we built it:

1. **Deployed the reference Validation Registry** (same ChaosChain reference repo) to Lightchain, initialized to point at our Identity Registry.
2. **Approved the relay as an operator** of agentId 1, so it can submit validation requests on the agent's behalf without ever holding the identity.
3. **Recorded one real end-to-end validation** for a genuinely stored file, so the registry isn't empty.

Verifiable values:

- **Validation Registry:** `0x4F880C5b75102620f70CB010f8d776538a340b49`
- **Deploy tx:** `0xb070b5d4921e7e56593487492bb639a5af1e7f3289675f9fc007900b3ab4df5a`
- **Wired to Identity Registry:** `getIdentityRegistry()` → `0x8cA5cc5037aF83762312b464e04426795d62CC40`
- **Sample requestHash:** `0xa3f9e0cf9699d0a1127ad4faee4aa1f33d2d858417b448743edf87a75311912b`
- **validationRequest tx:** `0x07441d4b3a81fdd3c49a2b5ad292ce809121324293a9a7723e4774cba0a3dd0d`
- **validationResponse tx:** `0x302f03016a42d4ccef859fcf3dfde593f818c952b7c30367f0166d0dd8e4860f`
- **Result:** `100` / tag `content-hash-match`
- **Validator wallet (v1):** `0xbb0ab4c9E15a20661CA0C4d2b6f5D32A7EdF7646`

Honest status (important): v1 validation is currently **self-attested** — the agent owner requests validation and the relay (same operations set) responds after re-reading the on-chain content hash. This is a real, standard-shaped validation artifact that indexers recognize, but it is **not** independent third-party validation. An outside validator can call `validationResponse` later with **zero contract changes** — that's the upgrade that turns "the rails are built" into "independently verified." Do not market this as "independently validated" until that happens.

Part 6 — Client feedback (ERC-8004 Reputation Registry)

This is the third and final registry.

What it is: the Reputation Registry is where any **client** (human or AI) who used the agent can leave structured, on-chain feedback — a numeric value plus optional tags. Raw signals live on-chain; scoring happens off-chain. **Critical rule built into the standard:** the feedback submitter **must not be the agent owner or operator** — the contract blocks self-reviews. So real reputation can only come from third parties who actually paid and used the service.

How we built it:

1. **Deployed the reference Reputation Registry** to Lightchain, initialized to point at our Identity Registry.
2. **Wired a skippable "rate this service (0-100)" step** so a paying client signs feedback with **their own wallet**.
3. The UI **blocks the agent owner and relay operator** from submitting through the app, matching the standard's anti-self-review rule.

Verifiable values:

- **Reputation Registry:** `0x62e7E1502Ef822FBfb66554f072b46DF8A8c08D0`
- **Deploy tx:** `0xcdbcb0c4e11eec6ea43b9967808932364318227caa579b4378224995cda0b53a`
- **Wired to Identity Registry:** `getIdentityRegistry()` → `0x8cA5cc5037aF83762312b464e04426795d62CC40`
- **Sample client feedback:** client `0xb85fB6Ad7A58Fa66E775920384CA61A2c854a0e9`, value **92**/100, tag `starred`, index `1`
- **Feedback tx:** `0xf3daa55a4bdb64b31737d775544a8164af87997d9ef16bfe3c1cb430043c99a0`

Honest status: the rails are live and any client wallet can leave feedback, and there's one genuine third-party rating (92). But a reputation score only means something once many real, independent clients rate the service. Reputation is **earned, not deployed** — don't claim strong verified reputation from a small or empty set of scores.

Part 7 — Discoverable from Ethereum (dual-registration on Base)

The registries above live on Lightchain, which Ethereum-side tools don't watch by default. So we also registered the **same agent** in the **canonical ERC-8004 Identity Registry on Base** — the official one the 8004 team deployed (at the same deterministic address on ~30 chains). No new contract to deploy; we just registered into the one that already exists.

How we did it:

1. Registered the agent (owned by the same agent wallet) into the canonical Base registry.
2. Updated the agent card's `registrations` array to list **both** the Lightchain and Base registrations — same card, same URL.
3. **Payment stays on Lightchain (2 LCAI or 1 bridged USDC)**. Dual-registration is about **discovery only**, not moving payments to Base.

Verifiable values:

- **Chain:** Base mainnet (**chain ID 8453**)
- **Canonical IdentityRegistry:** `0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`
- **Base agent ID:** `73323`
- **Register tx:** `0x770c431116021215629f1c07229ac742538e4158c8fca4d18ec6d1033ca6401a`
- **Owner:** `0xBF78164b6626ea8096C4B6b268b0b70f86f66893`
- **agentRegistry (CAIP format):** `eip155:8453:0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`

The net effect: an Ethereum-side agent can discover OrcaVault natively through a registry it already watches, and pay for it in LCAI or bridged USDC on Lightchain (where gas is near-zero). Identity is advertised on both chains; the paid service and its verifiable record stay on Lightchain.

Part 8 — Verify it yourself (quick reference)

Everything here is public and readable on-chain. Nothing below is a secret or a personal identifier.

Lightchain (chain ID 9200):

Piece	Address / value
Identity Registry	<code>0x8cA5cc5037aF83762312b464e04426795d62CC40</code>
Validation Registry	<code>0x4F880C5b75102620f70CB010f8d776538a340b49</code>
Reputation Registry	<code>0x62e7E1502Ef822FBfb66554f072b46DF8A8c08D0</code>
Agent ID	<code>1</code>
Agent (NFT) owner	<code>0xBF78164b6626ea8096C4B6b268b0b70f86f66893</code>
Payment address (2 LCAI or 1 USDC)	<code>0xbb0ab4c9E15a20661CA0C4d2b6f5D32A7EdF7646</code>
Bridged-USDC token (Lightchain)	<code>0x1113E23397b4398eF1A46A9A7f27C2241527b003</code>
Agent card	<code>https://orcavault.win/agents/orcavault-mcp-card.json</code>

Base (chain ID 8453):

Piece	Address / value
Canonical Identity Registry	0x8004A169FB4a3325136EB29fA0ceB6D2e539a432
Base agent ID	73323

How to check: look up any contract address on a Lightchain (or Base) block explorer, read `ownerOf(1)` on the Identity Registry, read `getIdentityRegistry()` on the Validation/Reputation registries to confirm they're wired together, or open the agent card URL to see the live JSON that ties it all together.

Summary — what OrcaVault actually achieved

- Wrapped a live, paid on-chain storage service as an **MCP tool** so AI agents can use it directly, with a pre-paid agent wallet that hides all wallet/gas/signing complexity.
- Added the **payment layer** ERC-8004 leaves out — pay in **2 LCAI or 1 bridged USDC**, on Lightchain.
- Deployed all **three ERC-8004 registries** — Identity, Validation, Reputation — on Lightchain, using the official reference contracts.
- **Dual-registered on Base** so Ethereum-side tools discover it natively.
- Kept every claim honest: identity is discovery not trust, validation is currently self-attested, and reputation is earned over time.

The point in one line: OrcaVault isn't chasing the "trust layer for AI" narrative — it's a working example of it, live today, and fully verifiable on-chain.

Reference implementation for all three registries: the ChaosChain `trustless-agents-erc-ri` repo (ERC-8004, Jan 2026). Standard: ERC-8004 "Trustless Agents," <https://eips.ethereum.org/EIPS/eip-8004>